Linux Kernel Programming

Linux Kernel programming

Agenda

- Packet Capture Handson
- Network debugging – tcpdump
- Network debugging – eBPF
- Process Management - General

Packet Capture - Handson

Network Debugging - TCPDump

- Protocol analyzers, also known as packet sniffers, capture network data for display or use by administrators. The captures include transport layer content, application layer information and header information, such as physical and logical addresses. Network administrators usually find this information more useful than the actual data payload.

- The following are three primary reasons network administrators capture packets:
  - **Security.** Captured packets expose insecure data, whether in the context of penetration testing or malicious eavesdropping attacks.
  - **Troubleshooting.** Header information is useful for troubleshooting everything from DNS to packet filters on routers.
  - **Auditing.** Captures display exactly what is on the network and not just what you think might be on the network. Audits verify that the expected types of network traffic are present.

- Tcpdump is a common open source Linux tool used to analyze packets. It's fast, straightforward and lightweight. Wireshark is a similar tool often used in conjunction with tcpdump. This article explores how to use tcpdump to capture and view packets. It also covers how to view tcpdump captures in Wireshark.

- The tcpdump utility runs on the Linux command line. Tcpdump is a simple application that works well in Linux servers without Linux-based network devices, a GUI or various IoT nodes. These attributes give tcpdump an advantage over more powerful GUI-based analyzers, like Wireshark. Tcpdump is also scriptable, which means it can enable scheduled captures.

Network Debugging - TCPDump

- On Ubuntu and similar distributions, type the following command:

#sudo apt-get install tcpdump

Start a capture

- To get started with tcpdump, type the following command in the Linux terminal:

tcpdump

Select an interface

- One way to narrow the capture is to specify the local network interface on which the analyzer captures. For example, captures are possible on a laptop's wired and wireless interfaces. Even more interfaces might be on network servers with multiple network interface cards or routers connected to several subnets. Use the -i option to select the interface. To display the available interfaces, type tcpdump -D.

tcpdump -i eth0

Network Debugging - TCPDump

Select host information

You probably have a good idea of what to look for in troubleshooting or penetration testing scenarios. You also likely know where the packets you need come from or go. Specify the source or destination IP addresses you want tcpdump to watch for with the following flags.

Flag	Explanation
`host`	Any packets with this host in the source or destination fields.
`src`	Any packets with this host in the source field.
`dst`	Any packets with this host in the destination field.
`src and dst`	Any packets with this host in both the source and destination fields.
`src or dst`	Any packets with this host either in the source field or destination field.

T

tcpdump -i eth0 host 10.1.1.42

If you want traffic that originates only from 10.1.1.42, type the following command:

tcpdump -i eth0 src host 10.1.1.42

Network Debugging - TCPDump

Filter by port number

- You might be more interested in a type of traffic than the hosts involved. In that case, use a filter based on port numbers. If you need Simple Mail Transfer Protocol traffic, type the following command:

tcpdump -i eth0 dst port 25

- If you want to find insecure web traffic, type the following command:

tcpdump -i eth0 dst port 80

Write the capture to a file

- One disadvantage of the examples above is tcpdump displays all results on the screen. They might roll by too quickly to analyze or detect patterns. It's usually better to write the capture results to a file instead.

- Use the -w option with a file name to specify a destination.

tcpdump -i eth0 -w capture.pcap

Network Debugging - TCPDump

View the capture file with tcpdump

- To view the file with tcpdump, type the following command:

tcpdump -r capture.pcap

Network Debugging - eBPF

- BPF Compiler Collection (BCC) is a library, which facilitates the creation of the extended Berkeley Packet Filter (eBPF) programs. The main utility of eBPF programs is analyzing the operating system performance and network performance without experiencing overhead or security issues.

- BCC removes the need for users to know deep technical details of eBPF, and provides many out-of-the-box starting points, such as the bcc-tools package with pre-created eBPF programs.

- Install the bcc-tools package, which also installs the BPF Compiler Collection (BCC) library as a dependency.

sudo apt-get install bpfcc-tools

<u>Displaying TCP connections added to the Kernel's accept queue</u>

- After the kernel receives the ACK packet in a TCP 3-way handshake, the kernel moves the connection from the SYN queue to the accept queue after the connection's state changes to ESTABLISHED. Therefore, only successful TCP connections are visible in this queue.

- The tcpaccept utility uses eBPF features to display all connections the kernel adds to the accept queue. The utility is lightweight because it traces the accept() function of the kernel instead of capturing packets and filtering them. For example, use tcpaccept for general troubleshooting to display new connections the server has accepted.

sudo tcpaccept-bpfcc

Network Debugging - eBPF

Tracing outgoing TCP connection attempts

- The tcpconnect utility uses eBPF features to trace outgoing TCP connection attempts. The output of the utility also includes connections that failed.

- The tcpconnect utility is lightweight because it traces, for example, the connect() function of the kernel instead of capturing packets and filtering them.

sudo tcpconnect-bpfcc

Measuring the latency of outgoing TCP connections

- The TCP connection latency is the time taken to establish a connection. This typically involves the kernel TCP/IP processing and network round trip time, and not the application runtime.

- The tcpconnlat utility uses eBPF features to measure the time between a sent SYN packet and the received response packet.

sudo tcpconnlat-bpfcc

Network Debugging - eBPF

Displaying details about TCP packets and segments that were dropped by the kernel

- The tcpdrop utility enables administrators to display details about TCP packets and segments that were dropped by the kernel. Use this utility to debug high rates of dropped packets that can cause the remote system to send timer-based retransmits. High rates of dropped packets and segments can impact the performance of a server.

- Instead of capturing and filtering packets, which is resource-intensive, the tcpdrop utility uses eBPF features to retrieve the information directly from the kernel.

sudo tcpdrop-bpfcc

Tracing TCP sessions

- The tcplife utility uses eBPF to trace TCP sessions that open and close, and prints a line of output to summarize each one. Administrators can use tcplife to identify connections and the amount of transferred traffic.

- For example, you can display connections to port 22 (SSH) to retrieve the following information:
  - The local process ID (PID)
  - The local process name
  - The local IP address and port number

Network Debugging - eBPF

- The amount of received and transmitted traffic in KB.
- The time in milliseconds the connection was active

#sudo tcplife-bpfcc -L 22

Tracing TCP retransmissions

- The tcpretrans utility displays details about TCP retransmissions, such as the local and remote IP address and port number, as well as the TCP state at the time of the retransmissions.

- The utility uses eBPF features and, therefore, has a very low overhead.

sudo tcpretrans-bpfcc

Displaying TCP state change information

- During a TCP session, the TCP state changes. The tcpstates utility uses eBPF functions to trace these state changes, and prints details including the duration in each state. For example, use tcpstates to identify if connections spend too much time in the initialization state.

sudo tcpstates-bpfcc

Network Debugging - eBPF

Displaying the network throughput by IP address and port

- The tcptop utility displays TCP traffic the host sends and receives in kilobytes. The report automatically refreshes and contains only active TCP connections. The utility uses eBPF features and, therefore, has only a very low overhead.

sudo tcptop-bpfcc

Tracing established TCP connections

- The tcptracer utility traces the kernel functions that connect, accept, and close TCP connections. The utility uses eBPF features and, therefore, has a very low overhead.

sudo tcptracer-bpfcc

Tracing IPv4 and IPv6 listen attempts

- The solisten utility traces all IPv4 and IPv6 listen attempts. It traces the listen attempts including that ultimately fail or the listening program that does not accept the connection. The utility traces function that the kernel calls when a program wants to listen for TCP connections.

sudo solisten-bpfcc

Network Debugging - eBPF

Summarizing the service time of soft interrupts

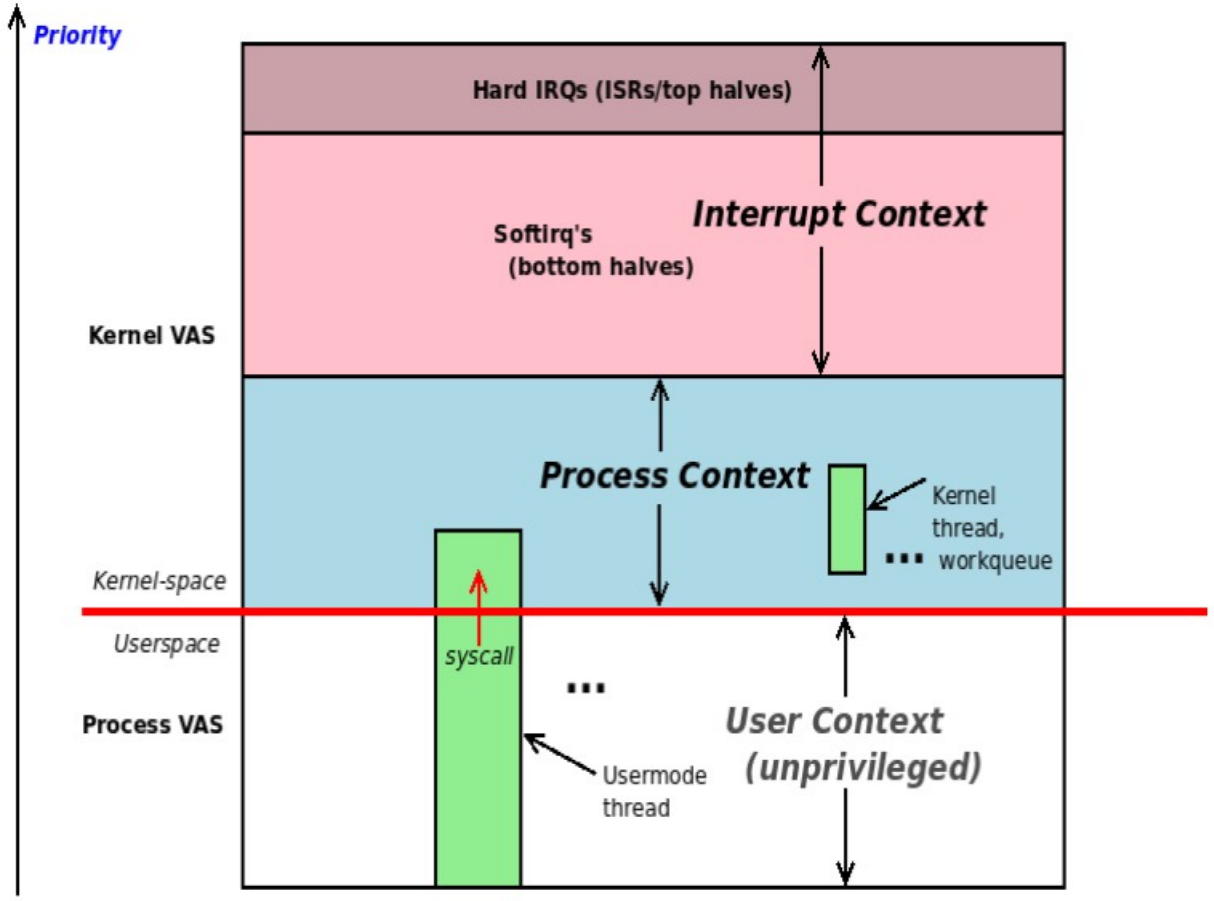
- The softirqs utility summarizes the time spent servicing soft interrupts (soft IRQs) and shows this time as either totals or histogram distributions. This utility uses the irq:softirq_enter and irq:softirq_exit kernel tracepoints, which is a stable tracing mechanism.

sudo softirqs-bpfcc

Process Management - Scheduler

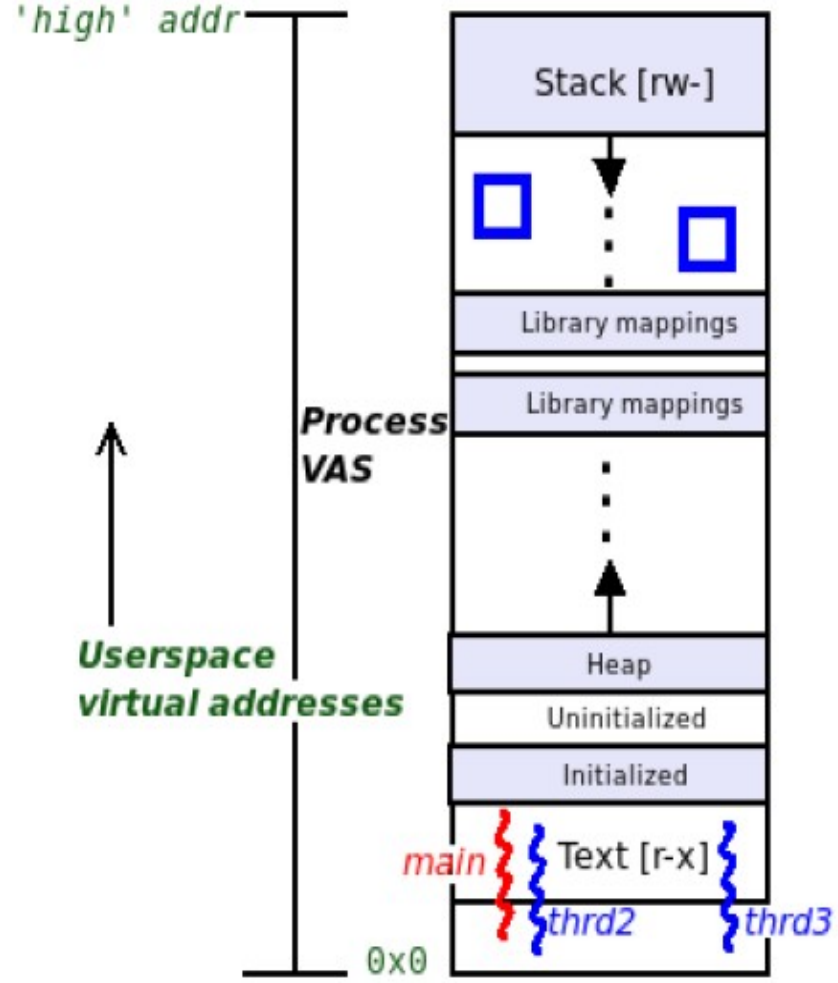
So, any and every piece of kernel code is entered by and executes in one of two contexts:

- Process context: The kernel is entered from a system call or processor exception (such as a page fault) and kernel code is executed, kernel data worked upon; it's synchronous (top down).
- Interrupt context: The is kernel entered from a peripheral chip's hardware interrupt and kernel code is executed, kernel data worked upon; it's asynchronous (bottom up).



Process Management - Scheduler

The user VAS is divided into homogeneous memory regions called segments or, more technically, mappings. Every Linux process has at least these mappings (or segments):

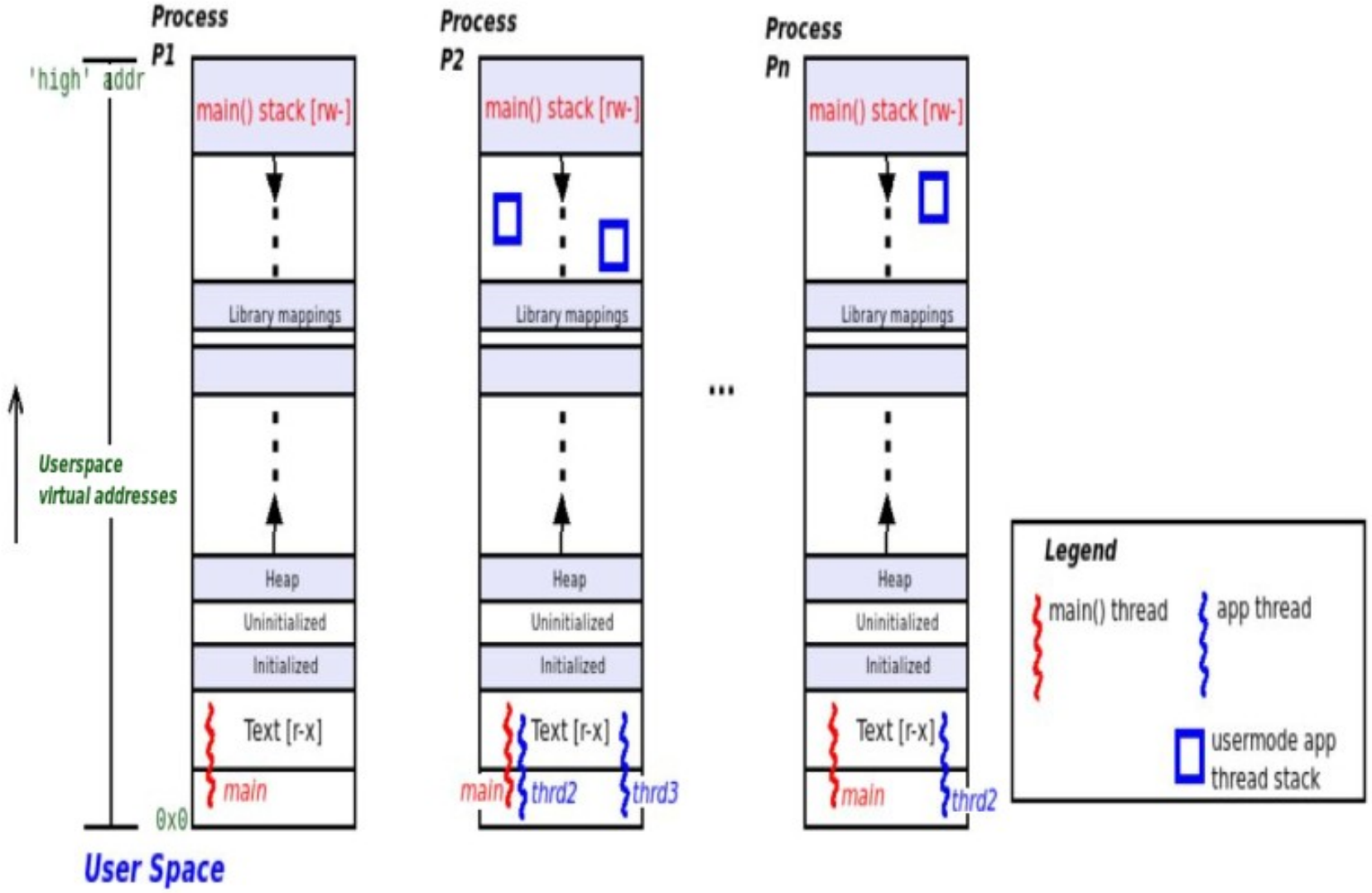


Process Management - Scheduler

In Linux, every user space thread alive has two stacks:

- **A user space stack:** This stack is in play when the thread executes user-mode code paths.
- **A kernel space stack:** This stack is in play when the thread switches to kernel mode (via a system call or processor exception) and executes kernel code paths (in process context).

Process Management - Scheduler



Process Management - Scheduler

- Every process has at least one thread of execution (the main() thread). In the preceding example, we show three processes P1, P2, and Pn, with one, three, and two threads in them respectively, including main().

- Each process consists of several segments (technically, mappings). Broadly, the user-mode segments (mappings) are as follows:
  - **Text**: Code; r-x
  - **Data segments**: rw-; consists of three distinct mappings – the initialized data segment, the uninitialized data segment (or bss), and an 'upward-growing' heap
  - **Library mappings**: For the text and data of each shared library the process dynamically links to
  - **Downward-growing stack(s)**

Process Management - Scheduler

- The following points can be derived for user space stack:

  - One user space stack is always present for the main() thread, it will be located close to the very top – the high end – of the user VAS; if the process is single-threaded (only a main()thread), then it will have just one user-mode stack; the P1 process in this case.

  - If the process is multithreaded, it will have one user-mode thread stack per thread alive (including main()); processes P2 and Pn in case. The stacks are allocated either at the time of calling fork(2) (for main()) or pthread_create(3) (for the remaining threads within the process), which results in this code path being executed in process context within the kernel:

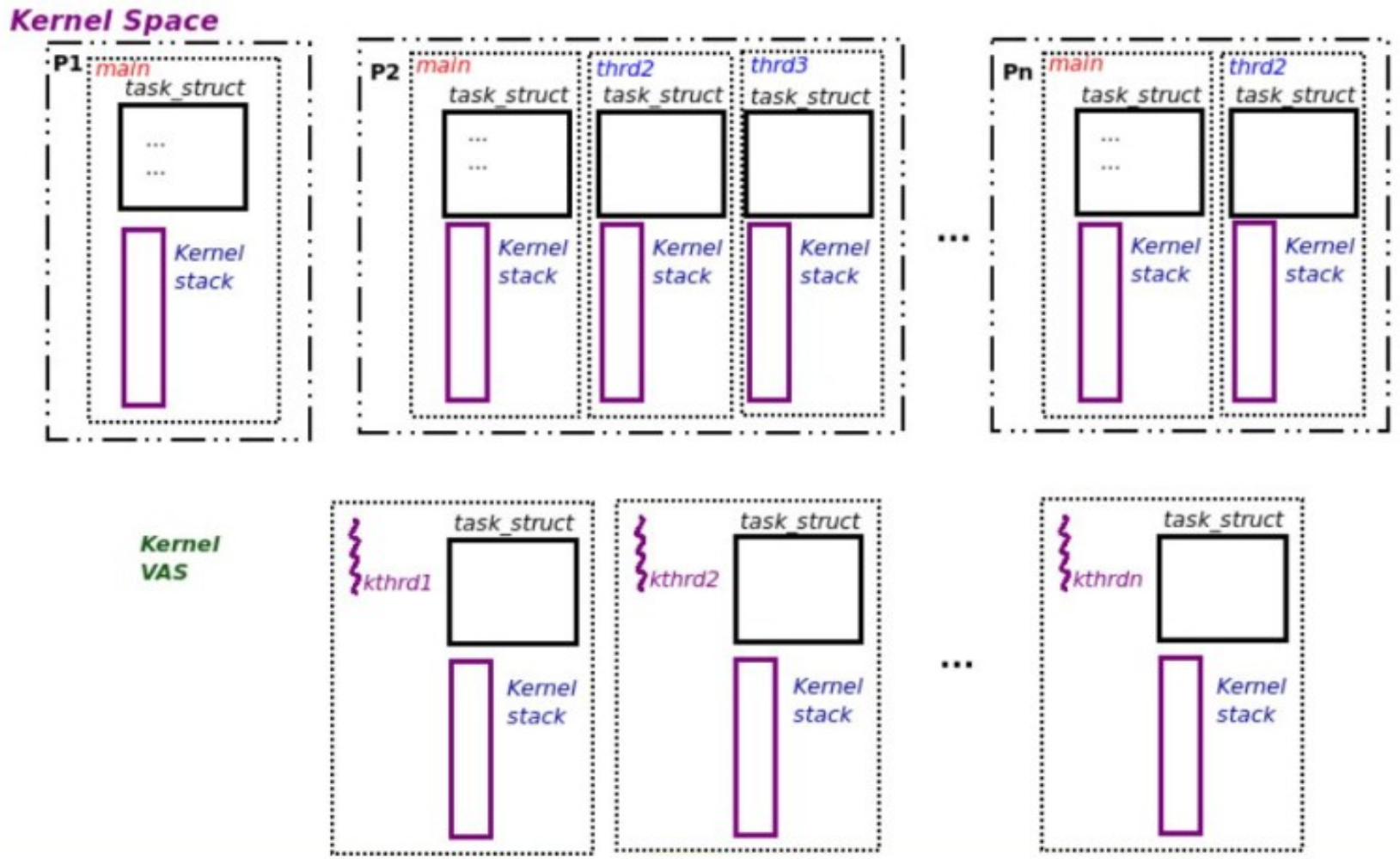
    sys_fork() --> do_fork() --> _do_fork()

  - The pthread_create(3) library API on Linux invokes the (very Linux-specific) clone(2) system call; this system call ends up calling _do_fork(); the clone_flags parameter passed along informs the kernel as to how exactly to create the '**custom process**'; in other words, a **thread**!

  - These user space stacks are of **course dynamic; they can grow/shrink up to the stack size resource limit** (RLIMIT_STACK, typically 8 MB; you can use the prlimit(1) utility to look it up).

Process Management - Scheduler

Kernel Space organization



Process Management - Scheduler

- The characteristics of kernel-mode stacks are:
  - There will be one kernel-mode stack for each application (user-mode) thread alive, including main().
  - Kernel-mode stacks are **fixed in size (static) and are quite small**.
  - Practically speaking, their size is 2 pages on 32-bit and 4 pages on 64-bit OSes (with a page typically being 4 KB in size).
  - They are allocated at thread creation time (usually boils down to _do_fork()).

Process Management - Scheduler

- each user-mode thread has two stacks – **a user-mode stack and a kernel-mode stack**. The exception to this rule is **kernel threads;** they only have a kernel-mode stack (as they possess no user mapping and thus no user space 'segments')

Kernel Space Stack

- For Viewing the kernel space stack of a given thread or process the powerful proc filesystem interfaces. Just peek under **/proc/<pid>/stack**.

```
$ sudo cat /proc/3085/stack
[<0>] do_wait+0x1cb/0x230
[<0>] kernel_wait4+0x89/0x130
[<0>] __do_sys_wait4+0x95/0xa0
[<0>] __x64_sys_wait4+0x1e/0x20
[<0>] do_syscall_64+0x5a/0x120
[<0>] entry_SYSCALL_64_after_hwframe+0x44/0xa9
$
```

- The process is currently executing the do_wait() function; it got there via a system call, the wait4() system call! This is quite right; the process works by forking off a child process and then waiting for its demise via the wait4(2) system call.

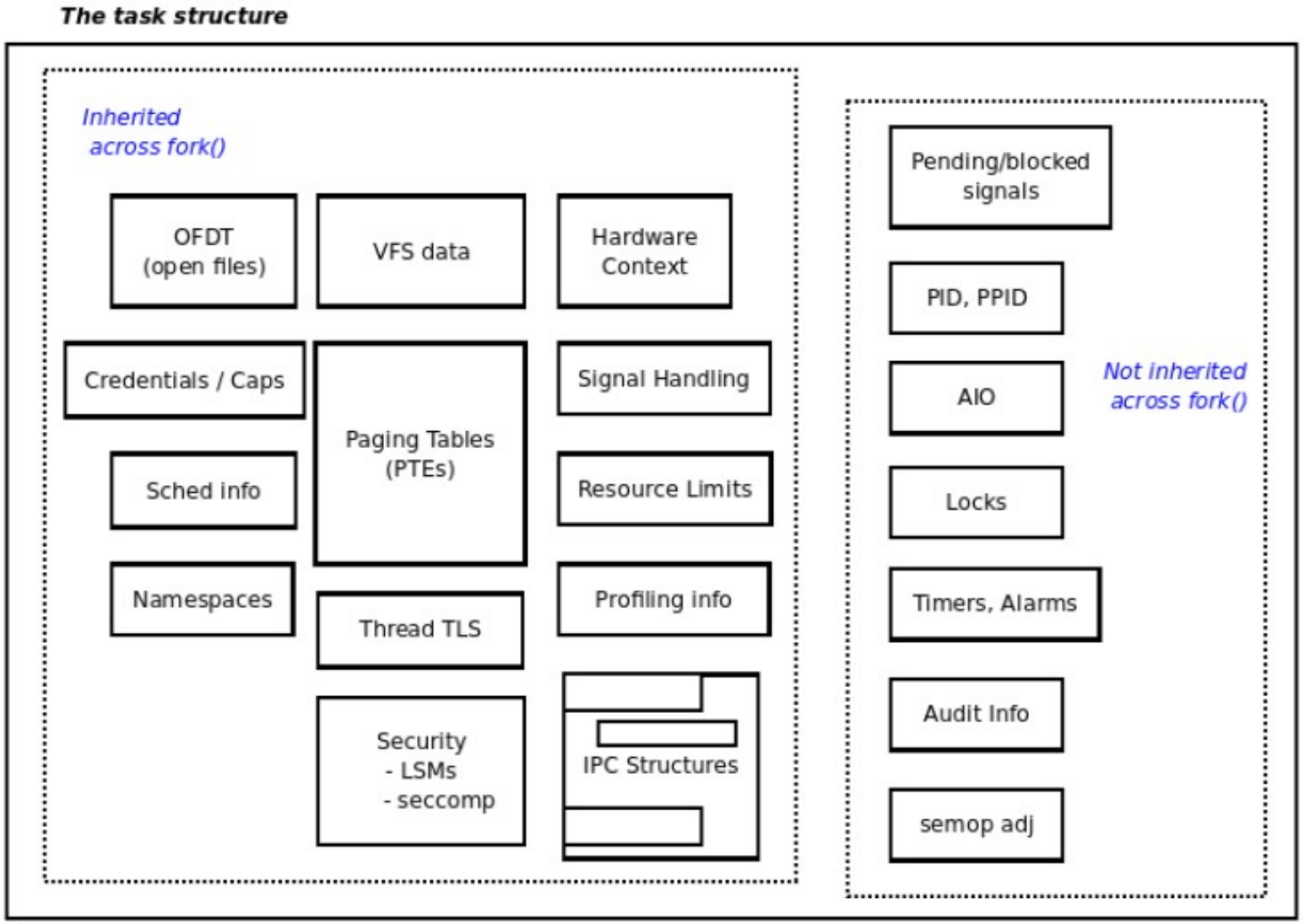
Process Management - Scheduler

User Space Stack

- The user space stack of a process or thread seems harder to do on a typical Linux distro (as opposed to viewing the kernel-mode stack, as we just saw in the previous section). There is a utility to do so: **gstack(1)**. In reality, it's just a simple wrapper over a script that invokes gdb(1) in batch mode, getting gdb to invoke its backtrace command.

- This can also be done using gdb directly by attaching to existing process and printing bt info.

Process Management - Scheduler



Process Management - Scheduler

```
// include/linux/sched.h
struct task_struct {
#ifdef CONFIG_THREAD_INFO_IN_TASK
    /*
     * For reasons of header soup (see current_thread_info()), this
     * must be the first element of task_struct.
     */
    struct thread_info      thread_info;    << important flags and status bits
>>
#endif
    /* -1 unrunnable, 0 runnable, >0 stopped: */
    volatile long           state;
    [...]
    void                *stack; << the location of the kernel-mode stack >>
    [...]
    /* Current CPU: */
    unsigned int cpu;
    [...]
```

Process Management - Scheduler

```
    [...]
    struct mm_struct *mm;       << memory management info >>
    struct mm_struct *active_mm;
    [...]
    pid_t pid;       << task PID and TGID values; explained below >>
    pid_t tgid;
    [...]
    /* Context switch counts: */
    unsigned long nvcsw;
    unsigned long nivcsw;
    [...]
    /* Effective (overridable) subjective task credentials (COW): */
    const struct cred __rcu *cred;
    [...]
    char comm[TASK_COMM_LEN];                << task name >>
    [...]
     /* Open file information: */
    struct files_struct *files;       << pointer to the 'open files' ds >>
    [...]
     /* Signal handlers: */
    struct signal_struct *signal;
    struct sighand_struct *sighand;
    sigset_t blocked;
    sigset_t real_blocked;
    [...]
#ifdef CONFIG_VMAP_STACK
    struct vm_struct *stack_vm_area;
#endif
    [...]
#ifdef CONFIG_SECURITY
    /* Used by LSM modules for access restriction: */
    void *security;
#endif
    [...]
    /* CPU-specific state of this task: */
```

Process Management - Scheduler

```
    struct thread_struct thread;        << task hardware context detail >>
    [...]
};
```

Process Management - Scheduler

Determining the context

- As you now know, kernel code runs in one of two contexts:
  - **Process (or task) context**
  - **Interrupt (or atomic) context**

- They are mutually exclusive – kernel code runs in either the process or atomic/interrupt context at any given point in time.

- Often, when writing kernel or driver code, it is imperative for you to first figure out what context the code that you're working on is running in. One way to learn this is by employing the following macro:

  #include <linux/preempt.h>

  in_task()

- It returns a Boolean: True if your code is running in process (or task) context, where it's – usually – safe to sleep; returning False implies you are in some kind of atomic or interrupt context where it is never safe to sleep.

Process Management - Scheduler

```
/* code: ch6/current_affairs/current_affairs.c */
[ ... ]
#include <linux/sched.h>      /* current */
#include <linux/cred.h>       /* current_{e}{u,g}id() */
#include <linux/uidgid.h>     /* {from,make}_kuid() */
[...]
#define OURMODNAME    "current_affairs"
[ ... ]

static void show_ctx(char *nm)
{
    /* Extract the task UID and EUID using helper methods provided */
    unsigned int uid = from_kuid(&init_user_ns, current_uid());
    unsigned int euid = from_kuid(&init_user_ns, current_euid());

    pr_info("%s:%s():%d ", nm, __func__, __LINE__);
    if (likely(in_task())) {
                pr_info(
                "%s: in process context ::\n"
                " PID          : %6d\n"
                " TGID         : %6d\n"
                " UID          : %6u\n"
                " EUID         : %6u (%s root)\n"
                " name         : %s\n"
                " current (ptr to our process context's task_struct) :\n"
                "            0x%pK (0x%px)\n"
 " stack start : 0x%pK (0x%px)\n",
                nm,
                /* always better to use the helper methods provided */
                task_pid_nr(current), task_tgid_nr(current),
                /* ... rather than the 'usual' direct lookups:
                    current->pid, current->tgid, */
                uid, euid,
                (euid == 0?"have":"don't have"),
                current->comm,
                current, current,
                current->stack, current->stack);
    } else
      pr_alert("%s: in interrupt context [Should NOT Happen here!]\n", nm);
}
```

Process Management - Scheduler

Differentiating between the process and thread – the TGID and the PID

- The Linux kernel uses a unique task structure (struct task_struct) to represent every thread, and as the unique member within it has a PID, this implies that, within the Linux kernel, every thread has a unique PID.

- This gives rise to an issue: how can multiple threads of the same process share a common PID? This violates the POSIX.1b standard (pthreads; indeed, for a while Linux was non-compliant with the standard, creating porting issues, among other things).

- To fix this annoying user space standards issue, Ingo Molnar (of Red Hat) proposed and mainlined a patch way back, in the 2.5 kernel series. A new member called the Thread Group IDentifier or TGID was slipped into the task structure.

- This is how it works: if the process is single-threaded, the tgid and pid values are equal. If it's a multithreaded process, then the tgid value of the main thread is equal to its pid value; other threads of the process will inherit the main thread's tgid value but will retain their own unique pid values.

Process Management - Scheduler

```
$ ps -LA
    PID   LWP  TTY          TIME  CMD
      1     1  ?        00:00:02  systemd
      2     2  ?        00:00:00  kthreadd
      3     3  ?        00:00:00  rcu_gp
[...]
    938   938  ?        00:00:00  VBoxService
    938   940  ?        00:00:00  RTThrdPP
    938   941  ?        00:00:00  control
    938   942  ?        00:00:00  timesync
    938   943  ?        00:00:03  vminfo
    938   944  ?        00:00:00  cpuhotplug
    938   945  ?        00:00:00  memballoon
    938   946  ?        00:00:00  vmstats
    938   947  ?        00:00:00  automount
 [...]
```

Process Management - Scheduler

- The ps(1) labels are as follows:
  - **The first column is PID** – this is actually representative of the tgid member of the task structure within the kernel for this task
  - **The second column is LWP (LightWeight Process or thread!)** – this is actually representative of the pid member of the task structure within the kernel for this task.

Process Management - CFS

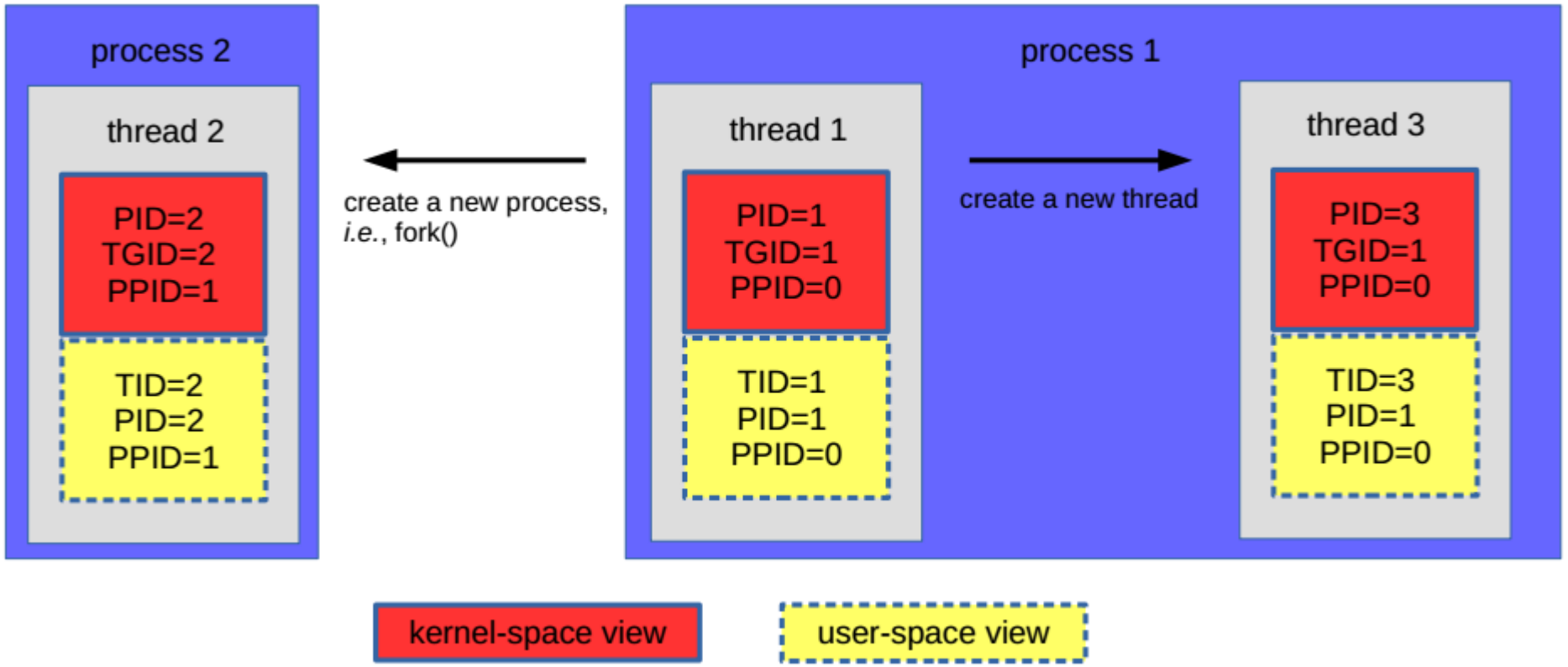
PID, TID, PPID, and TGID

- The smallest scheduling entity in the Linux kernel is a thread, not a process. Each thread is assigned a unique number for this purpose. In the kernel terms, confusingly enough, this number is called a process ID (PID). The process that threads belong to is accounted for as the thread group ID (TGID).

- When a new process starts by invoking fork(), it is assigned a new TGID. This newly forked process is created with a single thread, whose PID is the same as the TGID. The parent's TGID is called a parent PID (PPID). If the thread creates another thread, the new thread gets a different PID, but the same TGID is taken over.

- In the mean time, some user space applications (e.g., ps) have a different (probably better) naming convention: the PID and the TGID in kernel-space view are, respectively, called a thread ID (TID) and a PID in these userland applications.

Process Management - CFS



Process Management - Scheduler

Linux Scheduling Policy

Scheduling policy	Key Points	Priority Scale
SCHED_OTHER or SCHED_NORMAL	Always the default; threads with this policy are non-real-time; internally implemented as a Completely Fair Scheduling (CFS) class . The motivation behind this schedule policy is fairness and overall throughput.	Real-time priority is 0; the non-real-time priority is called the nice value: it ranges from -20 to +19 (a lower number implies superior priority) with a base of 0
SCHED_RR	The motivation behind this schedule policy is a (soft) real-time policy that's moderately aggressive. Has a finite timeslice (typically defaulting to 100 ms). A SCHED_RR thread will yield the processor IFF (if and only if): ❑ It blocks on I/O (goes to sleep). ❑ It stops or dies. ❑ A higher-priority real-time thread becomes runnable (which will preempt this one). ❑ Its timeslice expires.	(Soft) real-time: 1 to 99 (a higher number implies superior priority)

Process Management - Scheduler

Linux Scheduling Policy

Scheduling policy	Key Points	Priority Scale
SCHED_FIFO	The motivation behind this schedule policy is a (soft) real-time policy that's (by comparison, very) aggressive. A SCHED_FIFO thread will yield the processor IFF: ❑ It blocks on I/O (goes to sleep). ❑ It stops or dies. ❑ A higher-priority real-time thread becomes runnable (which will preempt this one). ❑ It has, in effect, infinite time slice.	(same as SCHED_RR)
SCHED_BATCH	The motivation behind this schedule policy is a scheduling policy that's suitable for non-interactive batch jobs, less preemption.	Nice value range (-20 to +19)
SCHED_IDLE	Special case: typically the PID 0 kernel thread (traditionally called the swapper; in reality, it's the per CPU idle thread) uses this policy. It's always guaranteed to be the lowest-priority thread on the system and only runs when no other thread wants the CPU.	The lowest priority of all (think of it as being below the nice value +19)

Process Management - Scheduler

- The priority scaling:

- Non-real-time threads (SCHED_OTHER) have a real-time priority of 0; this ensures that they cannot even compete with real-time threads. They use an (old UNIX-style) priority value called the nice value, which ranges from -20 to +19 (-20 being the highest priority and +19 the worst).

The way it's implemented on modern Linux, each nice level corresponds to an approximate 10% change (or delta, plus or minus) in CPU bandwidth, which is a significant amount.

- Real-time threads (SCHED_FIFO / SCHED_RR) have a real-time priority scale from 1 to 99, 1 being the least and 99 being the highest priority. Think of it this way: on a non-preemptible Linux system with one CPU, a SCHED_FIFO priority 99 thread spinning in an unbreakable infinite loop will effectively hang the machine!

Process Management - Scheduler

Analyzing using perf

sudo perf top

sudo perf top --sort comm,dso

sudo perf top -r 90 --sort pid,comm,dso,symbol

alias ptopv='sudo perf top -r 80 -f 99 --sort pid,comm,dso,symbol --demangle-kernel -v --call-graph dwarf,fractal'

sudo perf timechart record

sudo perf timechart

Process Management - Scheduler

- Ingo Molnar, a key kernel developer, (along with others) redesigned the internal structure of the kernel scheduler, introducing a new approach called scheduling classes (this was back in October 2007 with the release of the 2.6.23 kernel).

- A layer of abstraction was introduced under the core scheduling code, the schedule() function. This layer under schedule() is generically called the scheduling classes and is modular in design.

- The basic idea is this: when the core scheduler code (encapsulated by the schedule() function) is invoked, understanding that there are various available scheduling classes under it, it iterates over each of the classes in a predefined priority order, asking each if it has a thread (or process) that requires scheduling onto a processor (how exactly, we shall soon see).

- As of the 5.4 Linux kernel, these are the scheduler classes within the kernel, listed in priority order, with the highest priority first:

Process Management - Scheduler

```
// kernel/sched/sched.h
[ ... ]
extern const struct sched_class stop_sched_class;
extern const struct sched_class dl_sched_class;
extern const struct sched_class rt_sched_class;
extern const struct sched_class fair_sched_class;
extern const struct sched_class idle_sched_class;
```

- There we have it, the five scheduler classes – stop-schedule, deadline, (soft) real time, fair, and idle – in priority order, highest to lowest.

- Threads that adhere to either the **SCHED_FIFO or SCHED_RR** scheduling policy, map to the **rt_sched_class** (for their sched_class within the task structure), all threads that are SCHED_OTHER (or SCHED_NORMAL) map to the **fair_sched_class**, and the idle thread (swapper/n, where n is the CPU number starting from 0) always maps to **idle_sched_class** scheduling class.

Process Management - Scheduler

- Every thread is associated with it's own unique task structure (task_struct); within the task structure, the **policy** member specifies the scheduling policy that the thread adheres to (typically one of SCHED_FIFO, SCHED_RR, or SCHED_OTHER).

- It's exclusive - a thread can only adhere to one scheduling policy at any given point in time (it can be changed though).

- Similarly, another member of the task structure, **struct sched_class**, holds the modular scheduling class that the thread belongs to (which is also exclusive).

- Both the scheduling policy and priority are dynamic and can be queried and set programmatically.

Process Management - Scheduler

- When the kernel needs to schedule, this is the essential call sequence:

```
schedule() --> __schedule() --> pick_next_task()
```

- The actual iteration over the preceding scheduling classes occurs here; see the (partial) code of pick_next_task(), as follows:

```
// kernel/sched/core.c
 /*
  * Pick up the highest-prio task:
  */
static inline struct task_struct *
pick_next_task(struct rq *rq, struct task_struct *prev, struct rq_flags
*rf)
{
    const struct sched_class *class;
    struct task_struct *p;

    /* Optimization: [...] */
    [...]

    for_each_class(class) {
        p = class->pick_next_task(rq, NULL, NULL);
        if (p)
            return p;
    }

    /* The idle class should always have a runnable task: */
    BUG();
}
```

Process Management - Scheduler

- The preceding **for_each_class()** macro sets up a for loop to iterate over all scheduling classes. Its implementation is as follows:

```
// kernel/sched/sched.h
[...]
#ifdef CONFIG_SMP
#define sched_class_highest (&stop_sched_class)
#else
#define sched_class_highest (&dl_sched_class)
#endif

#define for_class_range(class, _from, _to) \
    for (class = (_from); class != (_to); class = class->next)

#define for_each_class(class) \
    for_class_range(class, sched_class_highest, NULL)
```

Process Management - Scheduler

- The preceding **for_each_class()** macro sets up a for loop to iterate over all scheduling classes. Its implementation is as follows:

```
// kernel/sched/sched.h
[...]
#ifdef CONFIG_SMP
#define sched_class_highest (&stop_sched_class)
#else
#define sched_class_highest (&dl_sched_class)
#endif

#define for_class_range(class, _from, _to) \
    for (class = (_from); class != (_to); class = class->next)

#define for_each_class(class) \
    for_class_range(class, sched_class_highest, NULL)
```

Process Management - Scheduler

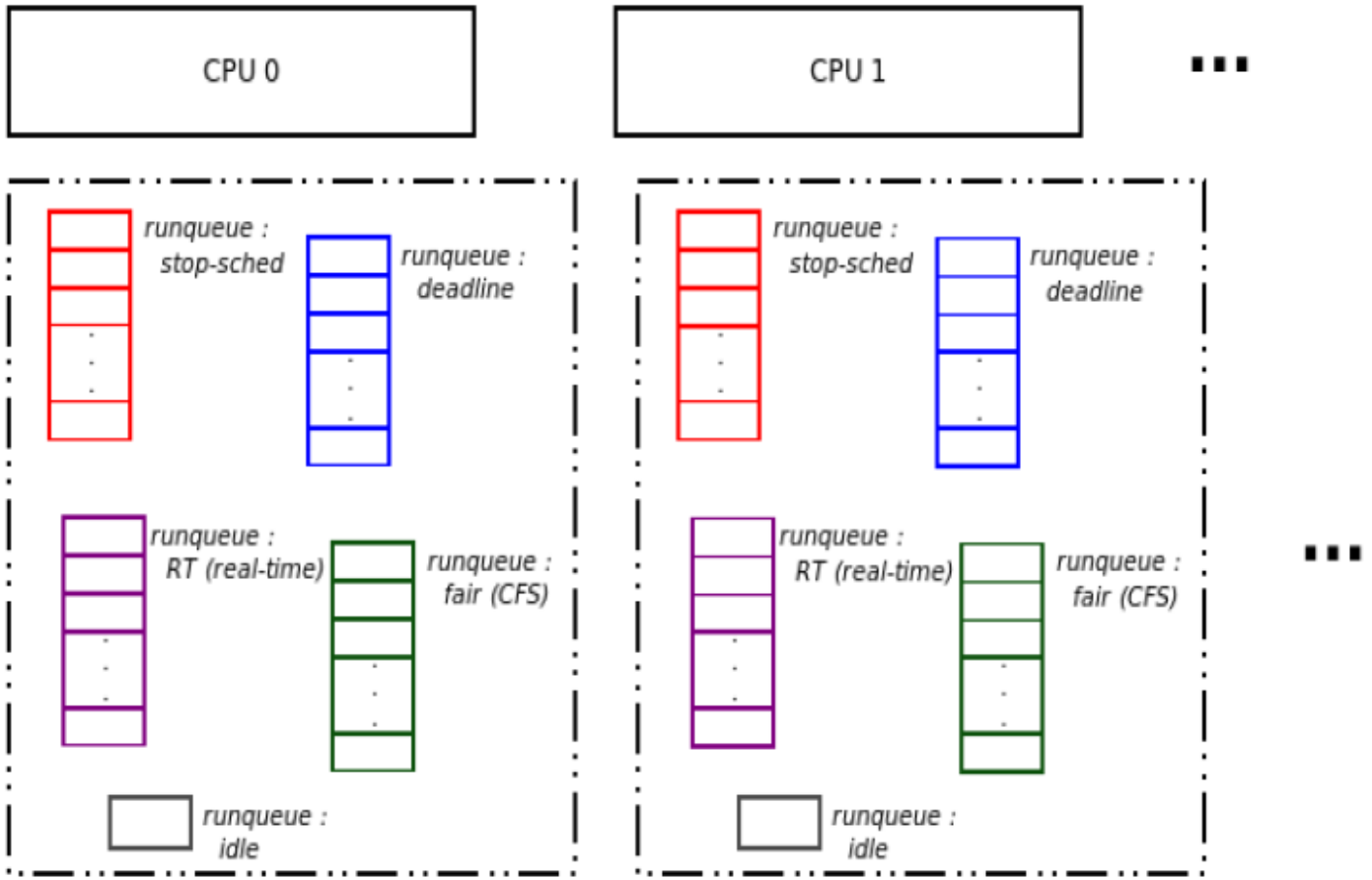
- We can see here that each sched_class has pointer to next class. As example see fair_sched_class.

```
// location: kernel/sched/sched.h
[ ... ]
struct sched_class {
    const struct sched_class *next;
    [...]
    void (*enqueue_task) (struct rq *rq, struct task_struct *p, int flags);
    void (*dequeue_task) (struct rq *rq, struct task_struct *p, int flags);
    [ ... ]
    struct task_struct * (*pick_next_task)(struct rq *rq,
                          struct task_struct *prev,
                          struct rq_flags *rf);
    [ ... ]
    void (*task_tick)(struct rq *rq, struct task_struct *p, int queued);
    void (*task_fork)(struct task_struct *p);
    [ ... ]
};
```

```
// kernel/sched/fair.c
const struct sched_class fair_sched_class = {
    .next = &idle_sched_class,
    .enqueue_task = enqueue_task_fair,
    .dequeue_task = dequeue_task_fair,
    [ ... ]
    .pick_next_task = pick_next_task_fair,
    [ ... ]
    .task_tick = task_tick_fair,
    .task_fork = task_fork_fair,
    .prio_changed = prio_changed_fair,
    [ ... ]
};
```

Process Management - Scheduler

- Now, this is a key point: the kernel maintains one runqueue for every processor core and for every scheduling class! So, if we have a system with, say, eight CPU cores, then we will have 8 cores * 5 sched classes = 40 runqueues! Runqueues are in fact implemented as per-CPU variables, an interesting lock-free technique (exception: on Uniprocessor (UP) systems, the stop-sched class does not exist):



Process Management - Scheduler

- To help better understand the scheduler class model, we will devise an example: let's say, on an Symmetric Multi Processor (SMP) or multicore) system, we have 100 threads alive (in both user and kernel space). Among them, we have a few competing for the CPUs; that is, they are in the ready-to-run (run) state, implying they are runnable and thus enqueued on runqueue data structures:
  - Thread S1: Scheduler class, stop-sched (SS)
  - Threads D1 and D2: Scheduler class, Deadline (DL)
  - Threads RT1 and RT2: Scheduler class, Real Time (RT)
  - Threads F1, F2, and F3: Scheduler class, CFS (or fair)
  - Thread I1: Scheduler class, idle.

Process Management - Scheduler

- Imagine that, to begin with, **thread F2** is on a processor core, happily executing code. At some point, the kernel wishes to context switch to some other task on that CPU (usually done by tick timer or voluntary context switches).

- On the scheduling code path, the kernel code ultimately ends up in the **kernel/sched/core.c:void schedule(void)** kernel routine .

- What's important to understand for now is that the pick_next_task() routine, invoked by schedule(), iterates over the linked list of scheduler classes, asking each whether it has a candidate to run.

- It’s code path (conceptually, of course) looks something like this:
  - Core scheduler code (schedule()): "Hey, SS, do you have any threads that want to run?"
  - SS class code: Iterates over its runqueue and does find a runnable thread; it thus replies: "Yes, I do, it's thread S1."
  - Core scheduler code (schedule()): "Okay, let's context switch to S1.“ And the job is done.

Process Management - Scheduler

- But what if there is no runnable thread S1 on the SS runqueue for that processor (or it has gone to sleep, or is stopped, or it's on another CPU's runqueue). Then, SS will say "no" and the next most important scheduling class, DL, will be asked. If it has potential candidate threads that want to run (D1 and D2, in our example), its class code will identify which of D1 or D2 should run, and the kernel scheduler will faithfully context switch to it.

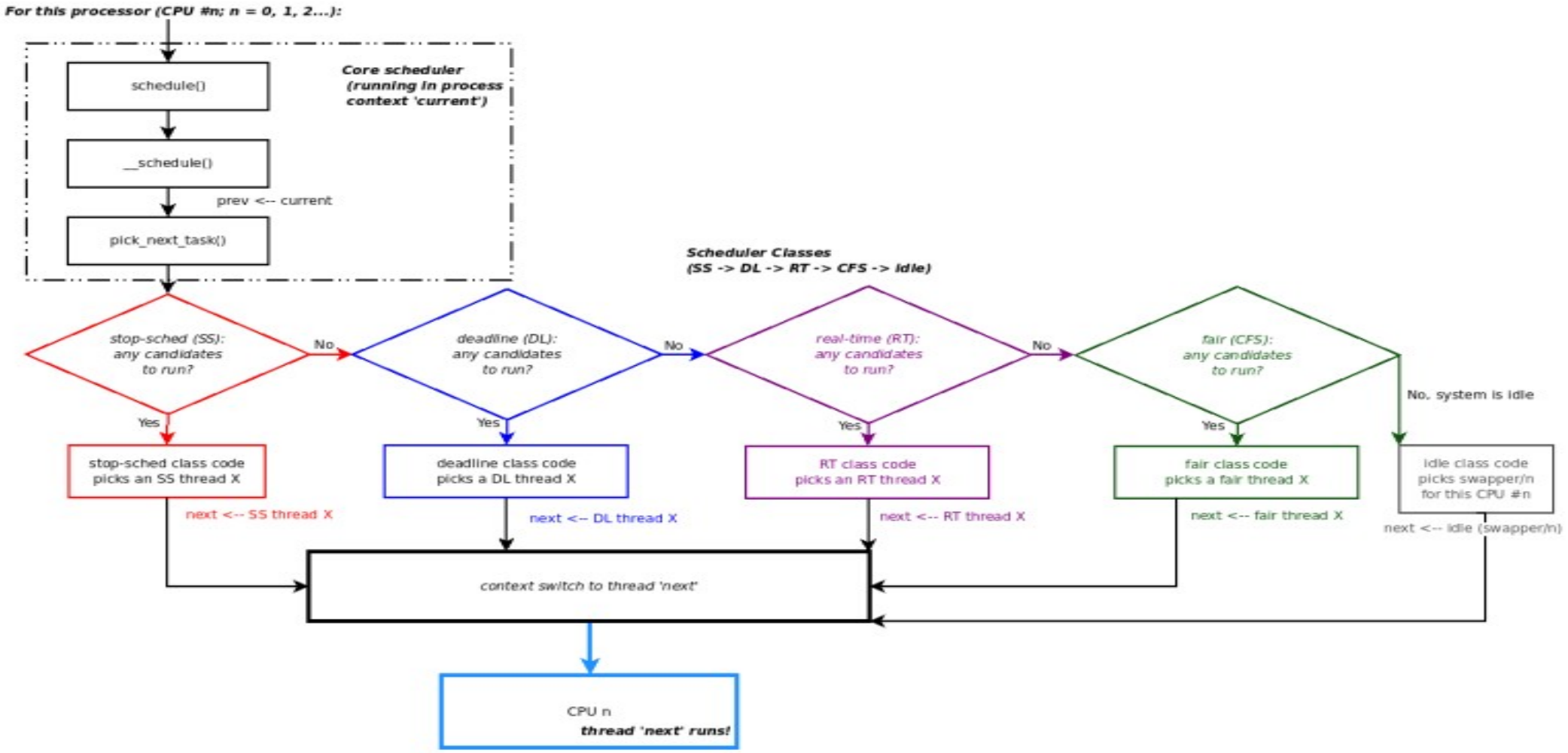
- This process continues for the RT and fair (CFS) scheduling classes.

- In all likelihood (on your typical moderately loaded Linux system), there will be no SS, DL, or RT candidate threads that want to run on the CPU in question, and there often will be at least one fair (CFS) thread that will want to run; hence, it will be picked and context-switched to.

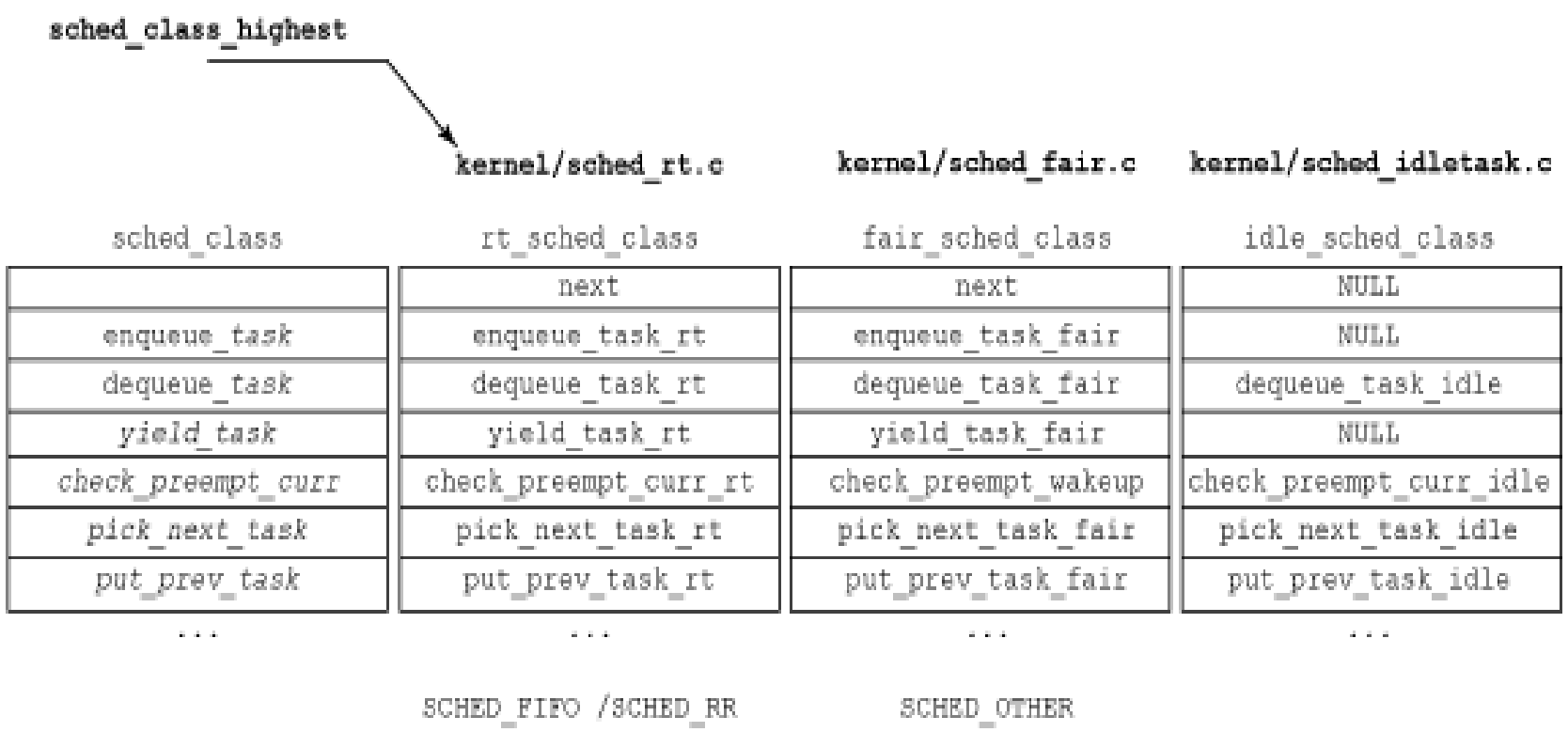
- If there's none that wants to run (no SS/DL/RT/CFS class thread wants to run), it implies that the system is presently idle (lazy chap). Now, the idle class is asked whether it wants to run: it always says yes! This makes sense: after all, it is the CPU idle thread's job to run on the processor when no one else needs to. Hence, in such a case, the kernel switches context to the idle thread (typically labelled swapper/n, where n is the CPU number that it's executing upon (starting from 0)).

Process Management - Scheduler

- The following diagram neatly sums up the way the core scheduling code invokes the scheduling classes in priority order, context switching to the ultimately selected next thread:



Process Management - CFS



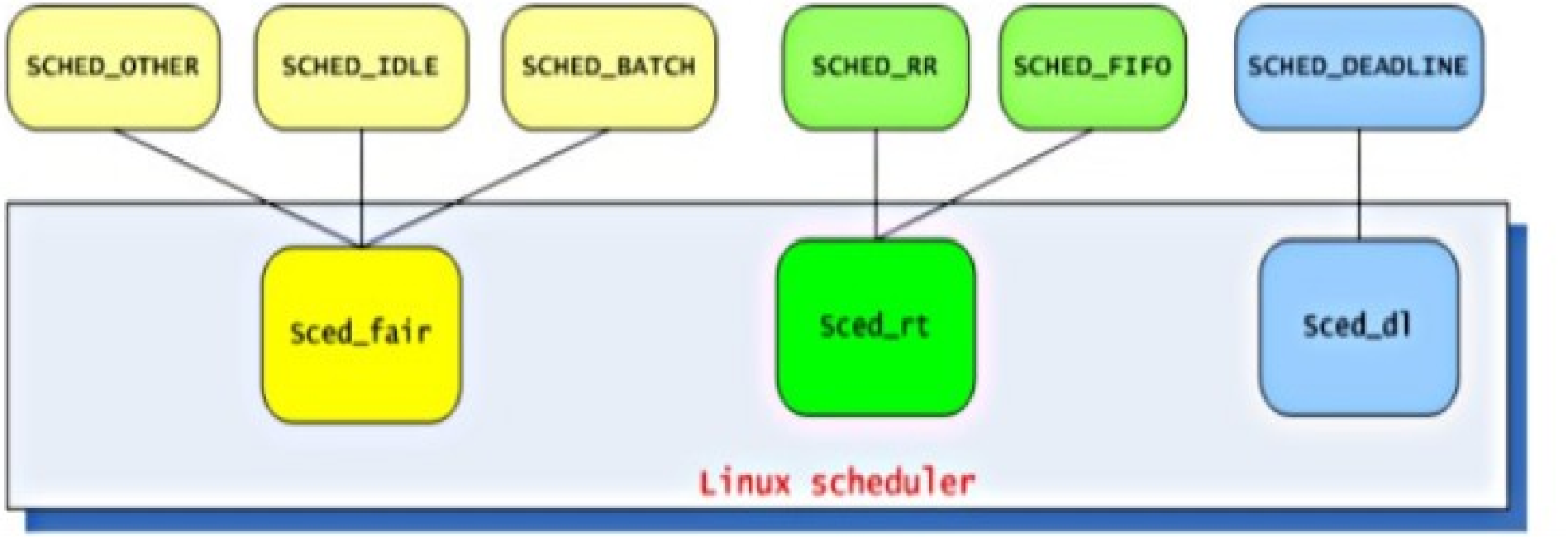
Process Management - CFS

- The kernel provides five schedulers by default, and the Linux kernel uses struct sched_class to abstract the scheduler:
  - Stop scheduler, stop_sched_class: the scheduling class with the highest priority, which can preempt all other processes and cannot be preempted by other processes;
  - Deadline scheduler, dl_sched_class: use the red black tree to sort the processes according to the absolute deadline, and select the smallest process for scheduling;
  - RT scheduler, rt_sched_class: real time scheduler, maintaining a queue for each priority;
  - CFS scheduler, cfs_sched_class: fully fair scheduler, which adopts fully fair scheduling algorithm and introduces the concept of virtual runtime;
  - Idle task scheduler, idle_sched_class: idle scheduler. Each CPU will have an idle thread. When no other process can be scheduled, the idle thread will be scheduled to run;

Process Management - CFS

- The Linux kernel provides some scheduling strategies for the user program to select the scheduler. The Stop scheduler and the idle task scheduler are only used by the kernel and cannot be selected by the user:
  - SCHED_DEADLINE: deadline process scheduling policy, which enables task to select deadline scheduler to schedule operation;
  - SCHED_RR: real time process scheduling strategy, time slice rotation. After the process runs out of time slice, it will be added to the tail of the run queue corresponding to the priority and give the CPU to other processes with the same priority;
  - SCHED_FIFO: real-time process scheduling strategy. First in first out scheduling has no time slice. When there is no higher priority, you can only wait to actively give up the CPU;
  - SCHED_NORMAL: ordinary process scheduling strategy, which enables task to select CFS scheduler to schedule operation;
  - SCHED_BATCH: ordinary process scheduling strategy, batch processing, so that task selects CFS scheduler to schedule operation;
  - SCHED_IDLE: ordinary process scheduling strategy, which enables task to select CFS scheduler with the lowest priority to schedule operation;

Process Management - CFS



Process Management - CFS

Scheduling classes and policies

- Stop
  - No policy
- Deadline
  - SCHED_DEADLINE
- Real Time
  - SCHED_FIFO
  - SCHED_RR
- Fair
  - SCHED_NORMAL
  - SCHED_BATCH
  - SCHED_IDLE
- Idle
  - No policy

Process Management - CFS

Run queues

- The core scheduler manages ready threads by enqueueing them into a run queue, which is implemented by struct rq.

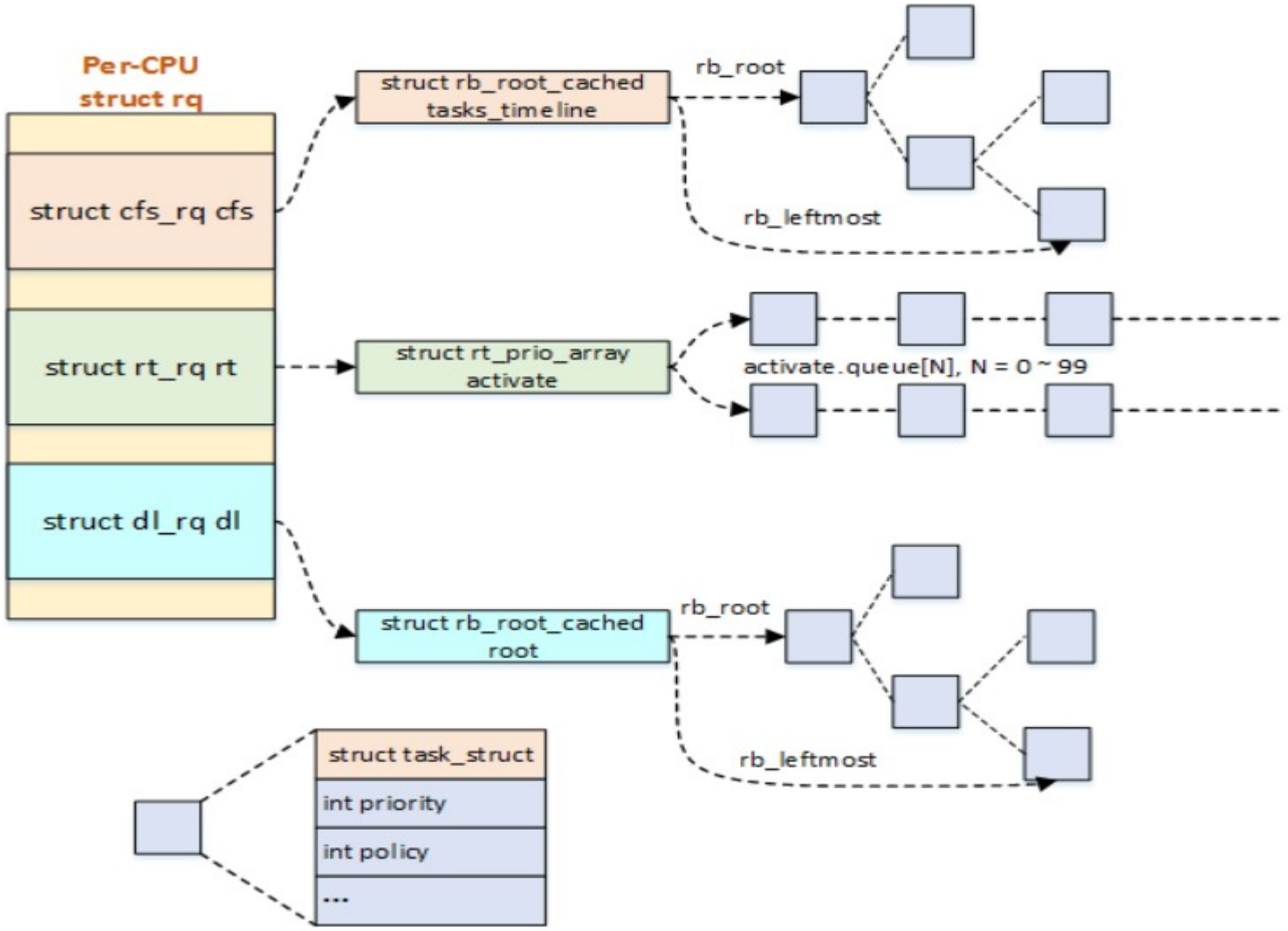
```
struct rq {
    ...
    struct cfs_rq cfs;
    struct rt_rq rt;
    ...
    struct task_struct *curr, *idle, *stop;
    ...
    u64 clock;
    ...
};
```

Process Management - CFS

- Each CPU has its own run queue, i.e., there are as many run queues as the number of CPUs in a system. A ready thread can belong to a single run queue at a time, since it is impossible that multiple CPUs process the same thread simultaneously. Here comes the need of load balancing among CPUs in multi-core systems. Without a special effort to balance load, threads may wait in a specific CPU's run queue, while other CPUs have nothing in their run queue, which, of course, means performance degradation.

- A run queue includes struct cfs_rq cfs and struct rt_rq rt, which are sub-run queues for the CFS and the RT scheduler, respectively. Enqueueing a thread into a run queue eventually means enqueueing it into either of these sub-run queues depending on the scheduler class of the thread. The thread that is currently running on a CPU is pointed by struct task_struct *curr defined in the run queue of the CPU. The per-run queue variable clock is used to store the latest time at which the corresponding CPU reads a clock source.

Process Management - CFS



Thank you